

Signal Object Detection - Machine Learning Project

Flavius-Ștefan Hăbeanu

Iunie 2026

Facultatea de Matematică și Informatică
Universitatea din București

Cuprins

1	Model K-NN	2
1.1	Prezentarea modelului	2
1.2	Descrierea modelului	2
1.3	Procesarea datelor și alegerea parametrilor	3
1.4	Concluzie	4
2	Model CNN	5
2.1	Prezentarea modelului și resursele folosite	5
2.2	Alegerea parametrilor și procesarea datelor	5
2.3	Metode folosite pentru prelucrare	9
2.4	Evoluția modelului și experimente eșuate	11
2.5	Concluzie	14

1 Model K-NN

1.1 Prezentarea modelului

Am ales algoritmul K-Nearest Neighbors ca o primă idee de rezolvare a acestei probleme de clasificare a semnalelor radio deoarece mi s-a părut cea mai intuitivă și la îndemână metodă de a obține un rezultat de început. Este un algoritm de învățare neparametric și de tip "lazy learning", adică nu construiește un model matematic în faza de antrenament, ci memorează tot setul de date.

Obiectivul modelului este să asocieze o spectrogramă uneia dintre cele cinci clase, căutând prin întregul set de antrenament cel mai similar semnal și aplicând un vot majoritar, pe baza vecinătății.

1.2 Descrierea modelului

Algoritmul calculează distanța dintre o spectrogramă de test și toate exemplele din train. Am utilizat distanța euclidiană L2 pentru a vedea asemănările între reprezentările vectoriale ale spectrogramelor. Modelul proiectează fiecare imagine într-un spațiu multidimensional.

Atunci când am un semnal nou, K-NN identifică cele mai apropiate K instanțe din setul de date, iar eticheta finală este decisă printr-un vot majoritar. Clasa care apare cel mai des printre vecinii selectați devine clasa modelului. Modelul devine extrem de sensibil la zgomotul din datele de antrenament și la calitate, dar e un proces simplu și ușor de înțeles.

1.3 Procesarea datelor și alegerea parametrilor

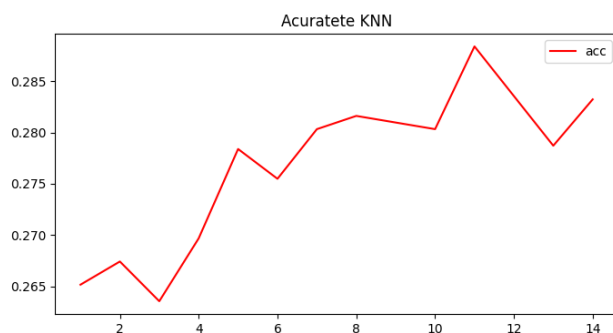


Figura 1: Acuratețe KNN

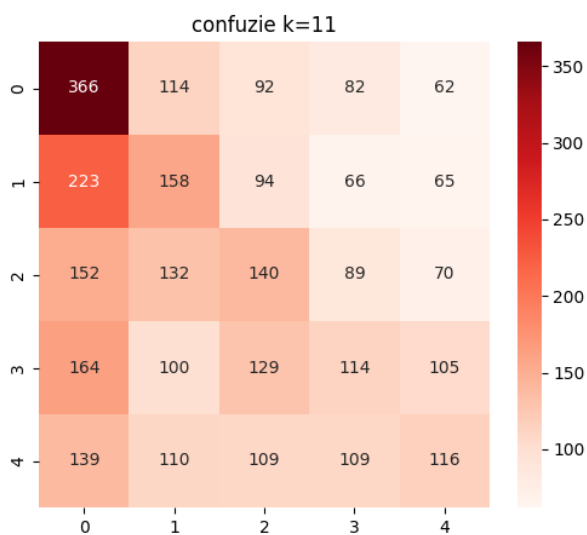


Figura 2: Matrice confuzie KNN

Am folosit o metodă bazată pe histogramme de culoare HSV deoarece este folositor să avem culoarea semnalului și este mai concret la variații mici de lumină decât RGB standard. Procesarea fiecărei imagini a presupus convertirea în spațiul HSV și calcularea histogrammei 3D cu 8 bin-uri pentru fiecare

canal (8x8x8). Am obținut un vector de 512 elemente pentru fiecare spectrogramă. Am utilizat distanța Manhattan L1 deoarece tinde să fie mai stabilă decât cea euclidiană. Pentru optimizarea performanței, am căutat cel mai bun K în intervalul [1,14] și am împărțit datele de antrenare în fold-uri cu raport 80/20.

```
def procesare(df,path):
    vec=[]
    for nume in df['id']:
        img=cv2.imread(os.path.join(path,nume)) # iau path-ul imaginii
        img_hsv=cv2.cvtColor(img,cv2.COLOR_BGR2HSV) # transform din bgr in hue, saturation, value
        #0=Hue, 1=Sat, 2=Value, impart in 8x8x8 elemente, ultimul interval e pt a include toate culorile posibile
        h=cv2.calcHist([img_hsv],[0,1,2],None,[8,8,8],[0,180,0,256,0,256])
        cv2.normalize(h,h)
        vec.append(h.flatten()) # flatten pt a transforma din 8x8x8 intr-un singur vector de 512 el
    return np.array(vec)

data_train=procesare(train,'train')
labels=train['label'].values
data_test=procesare(test,'test')

#fac split pe date 80-20
X_tr,X_val,y_tr,y_val=train_test_split(data_train,labels,test_size=0.2,random_state=67)
```

Figura 3: Procesarea datelor KNN

1.4 Concluzie

Modelul K-NN este o soluție de bază eficientă pentru a înțelege cum se distribuie datele în spațiul HSV. Deși am obținut o metodă rapidă de clasificare prin histograme, performanța este una slabă pentru semnalele radio, acestea fiind mai complexe. Matricea de confuzie arată că modelul nu e capabil să distingă clar între clasele cu culori similare, unde spațiul vectorilor de 512 elemente nu a fost suficient pentru a surprinde alte detalii fine care să facă diferența. Totuși, este o soluție simplă și are o viteză de execuție ridicată, ceea ce constituie un punct bun de plecare.

2 Model CNN

2.1 Prezentarea modelului și resursele folosite

Pentru acest model care a obținut o precizie considerabil mai bună decât modelul anterior K-NN, am construit o arhitectură de rețea convoluțională CNN de la zero în PyTorch (numită în cod ModelProiect). Logica din spate a fost una simplă: m-am bazat pe faptul că semnalele primite sunt niște linii subțiri pe un fundal complet negru. De aceea nu am folosit doar filtre convoluționale pătrate obișnuite, ci am creat blocuri asimetrice (cu dimensiunile 1x7 și 7x1 față de cele 3x3 clasice, am creat eu practic scannere verticale și orizontale cu aceste dimensiuni pentru a detecta liniile).

În ceea ce privește resursele utilizate, am rulat modelul pe un calculator ce folosește o placă video NVIDIA RTX 3060 cu suport CUDA. Am căutat pe internet dacă este posibil să reduc timpul de execuție al unei epoci, având în vedere că placa video nu este atât de performantă pentru o extindere și mai mare a modelului, așa că m-am informat din acest articol (click) că este posibil acest lucru prin implementea tehnicii Automatic Mixed Precision (AMP), apoi în implementare m-am informat din documentația oficială (click) . Astfel, am redus pe cât posibil calculele pe 32 biți, care au fost făcute pe 16 biți.

O epocă a durat în jur de 200-210 secunde. Am folosit 5 fold-uri, fiecare de câte 90 epoci. La un scurt calcul, programul a rulat undeva la 25 de ore pe calculator.

2.2 Alegerea parametrilor și procesarea datelor

```
DIM=224
NR_FOLDURI=5
EPOCI=90
BATCH=16
LR=1e-3
WD=2e-2
a_mixup=0.2
p_mixup=0.35
SEED=67
```

Figura 4: Parametri folosiți

Așa cum se observă în poză, am ales dimensiunea imaginilor DIM=224, iar dimensiunea lotului BATCH=16. Acestea au fost alese strict pe baza capacității plăcii mele video de rulare, așa cum am zis și în subsecțiunea anterioară. La rezoluții mai mici (ca 192x192), am observat că confuzia modelului între clasele 4 și 5 apare din cauza unei linii fine care se estompează și este confundată în clasa 4 cu o linie verticală, specifică clasei 5. Deoarece am crescut foarte mult rezoluția, până la 224x224, a trebuit să limitez batch-ul la 16 imagini / iterație, altfel primeam eroarea Out of Memory.

Numărul de epoci l-am ales ca 90, perfect pentru OneCycleLR să-și construiască rata de învățare ca o curbă. Datele le-am împărțit în 5 fold-uri, iar SEED=67 este doar pentru a putea reproduce rezultatele la fiecare rulare de cod.

Pentru optimizare, am ales algoritmul AdamW cu rata de învățare LR=1e-3 (rata basic pentru OneCycleLR) și penalizarea Weight Decay WD=2e-2. Valoarea aceasta m-a ajutat să penalizez ponderile prea mari, forțând rețeaua să se concentreze pe semnal.

În ceea ce privește parametrii de augmentare MixUp alpha (a_mixup=0.2) și probabilitatea de aplicare (p_mixup=0.35), i-am ales așa deoarece tehnica aceasta combină matematic două imagini și etichetele lor, forțând modelul să fie mai sigur pe decizie. Dacă foloseam un alpha prea basic (cum ar fi 1.0 sau 0.4), imaginile se combinau prea echilibrat și transforma spectogramele într-o chestie gri în care o linie suprapusă peste alta nu putea fi recunoscută. Am setat alpha destul de mic, astfel încât la combinarea a două imagini, una rămâne vizual mai clară. Aplicând tehnica asta doar în 0.35 din cazuri, modelul a putut vedea datele în forma lor nealterată în 0.65 din cazuri, ceea ce a fost perfect pentru a extrage corect semnalul.

```

def cit_img(cale):
    im=Image.open(cale).convert('L').resize((DIM, DIM), Image.BILINEAR)
    img=np.asarray(im, dtype=np.float32)
    return (img-img.mean()/(img.std()+1e-8))

def citeste_set(csv, folder, cu_label=True):
    df=pd.read_csv(csv)
    nume=df['id'].tolist()
    lista_x=[]
    for nm in nume:
        lista_x.append(cit_img(os.path.join(folder, nm)))
    X=np.array(lista_x, dtype=np.float32)
    if cu_label==True:
        y=np.array([int(v) for v in df['label']], dtype=np.int64)
        return X, y, nume
    return X, nume

```

Figura 5: Citire imagini și set

Pentru procesarea datelor, mai întâi am construit prima metodă din imagine care preia o singură imagine și o pregătește pentru model. Astfel, mai întâi deschide imaginea de la locația cale, o convertește la formatul de gri (cu "L"), apoi o redimensionează la (224x224) folosind metoda Image.BILINEAR care mă ajută atunci când redimensionez mai jos imagini să umple spațiul suplimentar de pixeli din imaginile noi. Această metodă ghicește ce nuanță de gri trebuie să aibă pixelii noi. După aceea, transformă imaginea într-o matrice NumPy cu valori reale. În final returnează standardizarea imaginii. Adunarea cu 1e-8 este pentru a evita împărțirea la zero.

Metoda citeste_set încarcă tabelul cu datele imaginilor, extrage coloana id (care are numele fișierelor imaginilor) și o convertește într-o listă. Apoi creez o listă goală inițial, unde adaug normalizarea fiecărei imagini. Prin array-ul X, creez un tensor (o listă NumPy) și verific clasa fiecărei imagini. Transform clasa fiecărei imagini în întreg și creez vectorul de etichete y, ca mai apoi să returnez toate datele. Dacă nu există coloana cu label-uri, returnez doar X (pixelii imaginii) și nume (listă cu numele fișierelor).

```

def shift_img(img, max_pix=8):
    dy=np.random.randint(-max_pix, max_pix+1)
    dx=np.random.randint(-max_pix, max_pix+1)
    rez=np.roll(img, (dy, dx), axis=(0, 1))
    if dy>0: rez[:dy, :]=0
    elif dy<0: rez[dy:, :]=0
    if dx>0: rez[:, :dx]=0
    elif dx<0: rez[:, dx:]=0
    return rez

def mask_img(img, mm=20):
    r=img.copy()
    h, w=r.shape[0], r.shape[1]
    if np.random.rand()<0.4:
        y=np.random.randint(0, h-mm)
        r[y:y+mm, :]=0
    if np.random.rand()<0.4:
        x=np.random.randint(0, w-mm)
        r[:, x:x+mm]=0
    return r

```

Figura 6: Shift și Mask pe imagini

Funcția `shift_img` mută imaginea pe verticală și orizontală cu un număr random de pixeli, pentru situațiile când semnalul nu este centrat pe mijlocul imaginii. Dacă `dy` va fi negativ, asta înseamnă o mutare în sus, altfel pentru `dy` pozitiv, o mutare în jos. Pe orizontală, dacă `dy` este negativ, mută la stânga, altfel la dreapta. Funcția `roll` este cea care face această mutare a pixelilor, însă pixelii care ies din imagine sunt reintroduși pe marginea opusă, ceea ce nu face neapărat o buclă. De aceea, dacă `dy<0` (imaginea a coborât), `rez` suprascrie cu 0 toate liniile de sus a matricei. Analog se face și pentru restul if-urilor.

Funcția `mask_img` taie random niște linii orizontale sau verticale din imagine, făcându-le negre, astfel încât modelul să poată identifica clase chiar și când lipsesc informații. Se copiază mai întâi imaginea originală, asigurându-mă că imaginea originală rămâne nemodificată prin funcția `copy`, apoi extrag din această copie înălțimea (`h`, echivalent cu linii) și lățimea (`w`, echivalent cu coloane) pentru a ști limitele pentru decuparea imaginii. În funcție de probabilitatea care pică în if-uri, mai exact în 0.4 din cazuri, variabila `y` alege o

coordonată random între 0 și h-mm (pentru limită) și ia toate liniile dintre y și y+mm pentru a le anula (le egalează cu 0). Asemănător se procedează și pe coloană.

```
class SetImagini(Dataset):
    def __init__(self, X, y=None, aug=False):
        self.X=X
        self.y=y
        self.aug=aug
    def __len__(self):
        return len(self.X)
    def __getitem__(self, i):
        img=self.X[i]
        if self.aug==True:
            if np.random.rand()<0.5: img=img[:, ::-1]
            if np.random.rand()<0.3: img=img[::-1, :]
            if np.random.rand()<0.5: img=shift_img(img, 8)
            if np.random.rand()<0.4: img=mask_img(img)
            img=np.ascontiguousarray(img)
        t=torch.from_numpy(img).unsqueeze(0)
        if self.y is None: return t
        return t, int(self.y[i])

def drop_path(x, p=0., train=False):
    if p==0. or train==False: return x
    keep=1-p
    shape=(x.shape[0], ) + (1, )*(x.ndim-1)
    m=(keep+torch.rand(shape, dtype=x.dtype, device=x.device)).floor_()
    return x.div(keep)*m

def mixup(x, y, alpha=a_mixup):
    lam=max(np.random.beta(alpha, alpha), 1-np.random.beta(alpha, alpha))
    perm=torch.randperm(x.size(0), device=x.device)
    return lam*x+(1-lam)*x[perm], y, y[perm], lam
```

Figura 7: SetImagini, droppath și mixup pe imagini

2.3 Metode folosite pentru prelucrare

```
class ConvBNReLU(nn.Module):
    def __init__(self, ci, co, k=3, s=1, g=1):
        super().__init__()
        pad=(k[0]//2, k[1]//2) if isinstance(k, tuple) else k//2
        self.conv=nn.Conv2d(ci, co, k, s, pad, groups=g, bias=False)
        self.bn=nn.BatchNorm2d(co)
    def forward(self, x):
        return F.relu(self.bn(self.conv(x)), inplace=True)
```

Figura 8: ConvBNReLU

```

class CoordSpat(nn.Module):
    def __init__(self, c, red=4):
        super().__init__()
        self.ph=nn.AdaptiveAvgPool2d((None, 1))
        self.pw=nn.AdaptiveAvgPool2d((1, None))
        mid=max(c//red, 8)
        self.c1=nn.Conv2d(c, mid, 1, bias=False)
        self.bn=nn.BatchNorm2d(mid)
        self.ch=nn.Conv2d(mid, c, 1, bias=False)
        self.cw=nn.Conv2d(mid, c, 1, bias=False)
    def forward(self, x):
        h, w=x.shape[2], x.shape[3]
        xh=self.ph(x)
        xw=self.pw(x).transpose(2, 3)
        y=F.relu(self.bn(self.c1(torch.cat([xh, xw], dim=2))), inplace=True)
        xh, xw=torch.split(y, [h, w], dim=2)
        return x*torch.sigmoid(self.ch(xh))*torch.sigmoid(self.cw(xw.transpose(2, 3)))

```

Figura 9: Coordonate spațiale

```

class BlocProcesare(nn.Module):
    def __init__(self, ci, co, s=1, dp=0., coord=False):
        super().__init__()
        self.dp=dp
        self.skip=(s==1 and ci==co)
        self.coord=coord
        hid=ci*4
        self.expand=ConvBNReLU(ci, hid, k=1) if ci!=hid else nn.Identity()
        self.conv_3x3=ConvBNReLU(hid, hid, k=3, s=s, g=hid)
        self.conv_1x7=ConvBNReLU(hid, hid, k=(1, 7), s=s, g=hid)
        self.conv_7x1=ConvBNReLU(hid, hid, k=(7, 1), s=s, g=hid)
        self.gap=nn.AdaptiveAvgPool2d(1)
        self.fc1=nn.Linear(hid, max(hid//4, 32), bias=False)
        self.relu1=nn.ReLU(inplace=True)
        self.fc2=nn.Linear(max(hid//4, 32), hid*3, bias=False)
        if coord: self.ca=CoordSpat(hid)
        self.proj_conv=nn.Conv2d(hid, co, 1, bias=False)
        self.proj_bn=nn.BatchNorm2d(co)
    def forward(self, x):
        out=self.expand(x)
        u1=self.conv_3x3(out)
        u2=self.conv_1x7(out)
        u3=self.conv_7x1(out)
        u=u1+u2+u3
        s_tensor=self.gap(u).view(u.size(0), -1)
        atentie=F.softmax(self.fc2(self.relu1(self.fc1(s_tensor))).view(s_tensor.size(0), 3, u.size(1)), dim=1)
        v=u1*atentie[:, 0, :, None, None]+u2*atentie[:, 1, :, None, None]+u3*atentie[:, 2, :, None, None]
        if self.coord: v=self.ca(v)
        out=self.proj_bn(self.proj_conv(v))
        if self.skip: out=drop_path(out, self.dp, self.training)+x
        return out

```

Figura 10: Blocul de procesare

```

class ModelProiect(nn.Module):
    def __init__(self, nr_cls, dpr=0.3):
        super().__init__()
        self.stem=ConvBNReLU(1, 48, s=2)
        self.blocks=nn.ModuleList([
            BlocProcesare(48, 48), BlocProcesare(48, 96, s=2), BlocProcesare(96, 96),
            BlocProcesare(96, 192, s=2), BlocProcesare(192, 192), BlocProcesare(192, 192, coord=True),
            BlocProcesare(192, 384, s=2, coord=True), BlocProcesare(384, 384, coord=True), BlocProcesare(384, 384, coord=True)
        ])
        rate_list=torch.linspace(0, dpr, 9).tolist()
        for i in range(len(self.blocks)): self.blocks[i].dp=rate_list[i]
        self.head=ConvBNReLU(384, 768, k=1)
        self.gap=nn.AdaptiveAvgPool2d(1)
        self.drop=nn.Dropout(0.4)
        self.fc=nn.Linear(768, nr_cls)
    def forward(self, x):
        x=self.stem(x)
        for b in self.blocks: x=b(x)
        return self.fc(self.drop(self.gap(self.head(x)).flatten(1)))

```

Figura 11: Modelul meu propriu-zis

2.4 Evoluția modelului și experimente eșuate

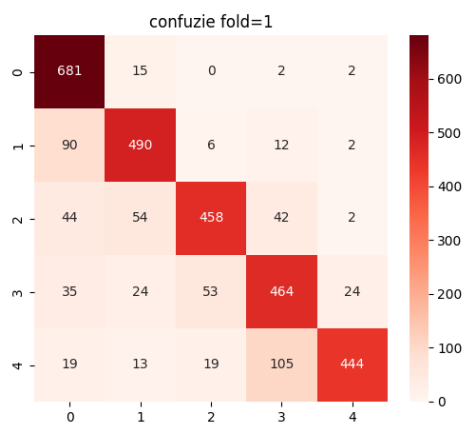


Figura 12: Confuzie fold 1

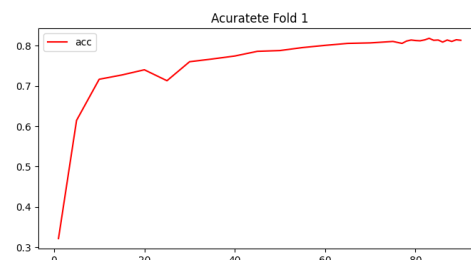


Figura 13: Acuratețe fold 1

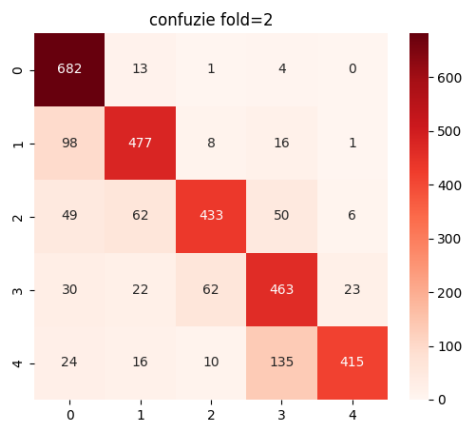


Figura 14: Confuzie fold 2

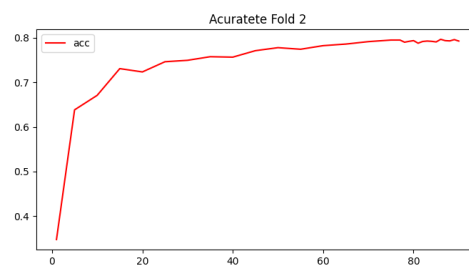


Figura 15: Acuratețe fold 2

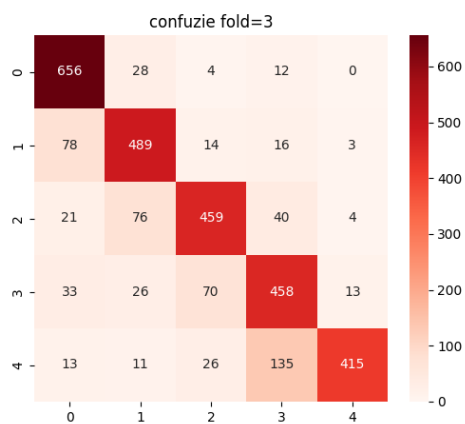


Figura 16: Confuzie fold 3

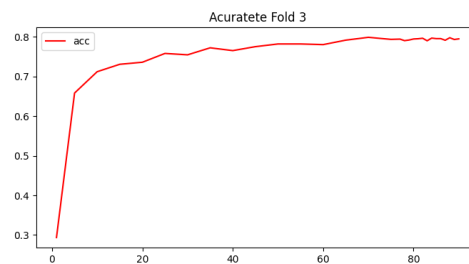


Figura 17: Acuratețe fold 3

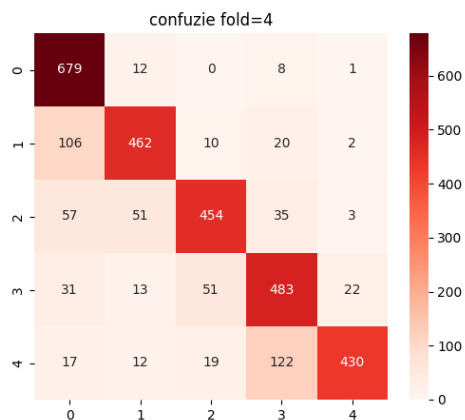


Figura 18: Confuzie fold 4

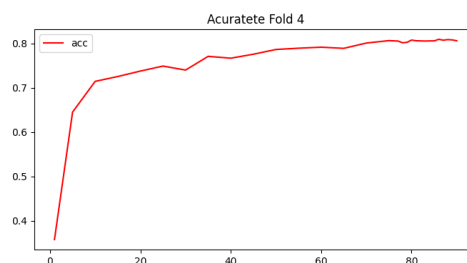


Figura 19: Acuratețe fold 4

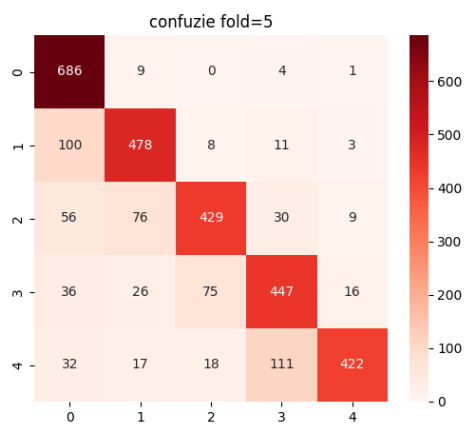


Figura 20: Confuzie fold 5

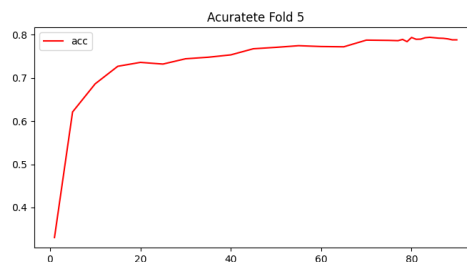


Figura 21: Acuratețe fold 5

La matricile de confuzie, deși indicii axelor sunt de la 0 la 4, aceștia corespund etichetelor reale ale claselor, care sunt cuprinse între 1 și 5.

Se observă cam în toate matricile de confuzie faptul că label-ul 5 este adesea confundat cu label-ul 4. Mai există o confuzie mare și la label-ul 2,

care de multe ori este confundat cu label-ul 1. Acuratețea crește considerabil până la 0.75, iar de acolo continuă să meargă în sus, însă nu cu aceeași viteză și cu mai multă grijă.

Înainte de a ajunge la soluția finală a acestui model, evident am mai încercat diverse modificări ale modelului. În primul rând, am testat dimensiunea filtrelor asimetrice. Inițial, testasem cu 3×3 , doar că acuratețea nu ajungea foarte mare deoarece rețeaua nu avea un câmp vizual suficient de mare pentru a prinde structura unui semnal. Apoi, am folosit filtre pătrate mari de dimensiuni 7×7 , însă chiar dacă vedeau o bucată mai mare din imagine, filtrele 7×7 aduceau un număr mare al parametrilor, ceea ce făcea rețeaua foarte lentă și ducea la overfit masiv, memorând zgomotul de pe fundal. Într-un final, m-am gândit să împart această dimensiune de 7×7 în două filtre paralele de 1×7 și 7×1 . Astfel, am același câmp vizual de care am nevoie, dar fără atât de multe calcule, făcând rețeaua rapidă.

Legat de rezoluția finală la care am ajuns (și anume 224×224), aceasta a fost aleasă după mai multe încercări nereușite (de fapt reușite, dar cu acuratețe mică) cu rezoluțiile 32×32 și 64×64 în primele zile când încă încercam să folosesc K-NN și SVM. Aș fi mărit această rezoluție și mai mult, însă limitările impuse de placa mea video nu mi-au permis acest lucru.

2.5 Concluzie

În concluzie, modelul CNN a demonstrat că este net-superior gestionării datelor comparativ cu K-NN. În timp ce modelul din urmă se baza pe distribuții cromatice, rețeaua neuronală a învățat structura și trăsăturile specifice fiecărei clase de semnale. Ajustarea rezoluției la 224×224 m-a ajutat să am mai puțină confuzie între clasele 4 și 5.

Tehnicile de augmentare precum MixUp și Mask m-au ajutat să fac un model mai bun, prevenind overfit-ul și generalizând mai bine datele. Deși configurarea hiperparametrilor și gestionarea memoriei plăcii video au fost cele mai mari provocări, algoritmul AdamW și optimizarea arhitecturii m-au ajutat să trec peste impas.